ISWC 2026 · HALF-DAY TUTORIAL

Façade-X: Querying Any Format as a Knowledge Graph

Integrate CSV, JSON, XML, spreadsheets and more using SPARQL alone.

📍 Bari, Italy · 📅 25 October 2026 · ⏱️ 14:10–18:00

SPARQL | Knowledge Graphs | Data Integration | Hands-on

The afternoon in one line

Any data format → a knowledge graph → SPARQL alone.

- The Façade-X idea and its foundations
- Hands-on with SPARQL Anything
- (coffee break)
- How it is used in practice, taught, and standardised

Agenda

Time	Session	Who
14:10–14:50	**Slot 1 · The Problem and the Idea**	
14:10–14:30	Introduction to Façade-X	Enrico
14:30–14:50	Foundations: towards a theory of façade-based data access	Luigi
14:50–16:10	**Slot 2 · Hands-on with SPARQL Anything**	
14:50–15:20	Live querying of CSV, JSON and XML	Luigi
15:20–15:45	Functions and magic properties	Luigi
15:45–16:10	Guided exercises with real datasets	Enrico
16:10–16:30	**Break · SPARQL Anything Clinic — bring your data**	**All**
16:30–18:00	**Slot 3 · From Practice to Standards**	
16:30–17:30	Industry talks: railway KG · TriplyDB deployment · Public Procurement Data Space	Mathias, Martin, Ivo
17:30–17:40	Teaching KG management with SPARQL Anything	Ryan
17:40–18:00	W3C Data Façades CG: roadmap and discussion	Enrico

Presenters

Luigi Asprino
Università Telematica San Raffaele, Roma (IT)

Co-founder of SPARQL Anything · chair, W3C Data Façades CG

Enrico Daga
KMi, The Open University (UK)

Co-founder of SPARQL Anything · chair, W3C Data Façades CG

Martin van Harmelen
Triply (NL)

SPARQL engine design and ELT pipelines on TriplyDB

Mathias Vanden Auweele
MATDATA (BE)

Railway infrastructure data and production KGs

Ryan Shaw
UNC Chapel Hill (US)

Teaches KG management with SPARQL Anything

Ivo Velitchkov
Kvistgaard (DK)

Consultant and trainer in semantic technologies

Show of hands ✋

Raise your hand if you…

1. have **heard of** Façade-X or SPARQL Anything before today
2. have **tried** SPARQL Anything, even once
3. **use** it in your work: research, a project, a product, teaching
4. have built a knowledge graph from CSV, JSON or XML **some other way**: RML/YARRRML, SPARQL Generate, scripts…

And: what data gives you the most trouble? Keep it for the clinic.

What you will learn today

IN PRACTICE

- Query **CSV, JSON, XML, Markdown, APIs** as a knowledge graph, with SPARQL alone
- Map any source to **your own vocabulary** in one `CONSTRUCT`
- **Join formats** in a single query
- Turn queries into **pipelines**: templates, parameters, reuse
- Use **SPARQL Anything** from the command line and from Python

BEYOND PRACTICE

- The **Façade-X idea**: one minimal model (containers, slots, values) for any format
- Its **theory**: what we can know about a query before touching the data
- How it is **deployed** in production: railways, procurement, TriplyDB
- How it is **taught** in KG courses
- How it is becoming a **standard**, and how to join the W3C CG

Before we start

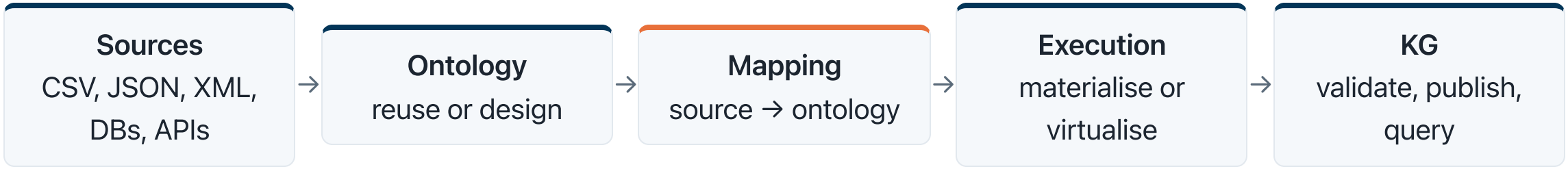
- **SPARQL Anything** downloaded, or **PySPARQL Anything** in Python / Jupyter
- No local setup? A **Google Colab** notebook is available
- Slides, notebooks and exercises: **w3c-facade-x.github.io/iswc2026-tutorial**
- Questions any time — and keep your own data for the clinic during the coffee break.

SLOT 1 · THE PROBLEM AND THE IDEA

Let's start.

w3c-facade-x.github.io/iswc2026-tutorial

Building a knowledge graph



Every method has these steps. The **mapping** is where most of the effort goes, and where the source formats get in the way.

Mapping today

- **One language per kind of source**: R2RML for databases, RML/YARRRML for files, XSPARQL for XML, SPARQL Generate…
- **A second language next to SPARQL**, with its own syntax, tools and debugging
- The mapping must **know the format**: JSONPath, XPath, column names, iterators
- Often the real answer is **a script** in Python or Java

The people who know the data are not always the people who can write the mapping.

The Façade-X idea

Don't map the format. Look at every format through the same façade, then map with SPARQL.

1. **Façade**: any source is seen as RDF with *one* generic structure, whatever its format
2. **Mapping**: a SPARQL `CONSTRUCT` from that structure to your ontology

The format-specific work moves into the engine. You only write SPARQL.

The meta-model: containers, slots, values

	What it is	In RDF
Container	anything that holds things: a file, a row, an object, an array, an element	a node (blank by default); the top one is `fx:root`
Slot	a position in a container, by **number** or by **name**	`rdf:_1`, `rdf:_2` … · `xyz:name`, `xyz:born` …
Value	the data	a literal
Type	an optional label of a container	`rdf:type`, e.g. an XML element name

That's all. No CSV-, JSON- or XML-specific vocabulary.

Same data, same façade

CSV (with headers)

```
name,born
Turner,1775
```

```
[] a fx:root ;
  rdf:_1 [
    xyz:name "Turner" ;
    xyz:born "1775" ] .
```

JSON

```
[ {"name": "Turner",
   "born": 1775} ]
```

```
[] a fx:root ;
  rdf:_1 [
    xyz:name "Turner" ;
    xyz:born 1775 ] .
```

XML

```
<artists>
  <artist born="1775">Turner</artist>
</artists>
```

```
[] a fx:root, xyz:artists ;
  rdf:_1 [ a xyz:artist ;
    xyz:born "1775" ;
    rdf:_1 "Turner" ] .
```

Three formats, one shape. Only the details differ: a JSON number stays a number, an XML element keeps its name as a type.

From the façade to your knowledge graph

```
CONSTRUCT {
  ?artist a schema:Person ; schema:name ?name ; schema:birthDate ?born .
} WHERE {
  SERVICE <x-sparql-anything:location=artists.csv,csv.headers=true> {
    ?row xyz:name ?name ; xyz:born ?born .
  }
  BIND (fx:entity(ex:, ?name) AS ?artist)
}
```

- `SERVICE <x-sparql-anything:…>` builds the façade of the source, on the fly
- Change `artists.csv` to `artists.json` : **the same query still works**
- Options in the IRI (or in the query) tune the façade: headers, delimiters, HTTP, slicing…

Why "opinionated"

- **Minimal on purpose**: four notions, so users learn one model, not one per format
- **SPARQL only**: no second language; joins, filters, functions come for free
- **Format-agnostic queries**: the shape of the data matters, not its syntax
- **Virtual or materialised**: query sources directly, or save the result as RDF
- The price: some format details are left out, by design

What comes next

- **Luigi**: what can we know about a query over a façade, before touching the data?
- **Hands-on**: SPARQL Anything on CSV, JSON, XML, and the guided exercises
- **After the break**: Façade-X in production, in teaching, and in the W3C CG